

Documentacion Final Actualizada - Proyecto 3

VeriLang en Rascal: parser, generador y sistema de tipos con TypePal

Curso: ISIS-2111 - Elementos Esenciales de Lenguajes de Programacion

Estudiante: Ronny Esteban Lopez Martinez

Codigo: 202415741

Proyecto Rascal: rascaldslverilang

Entrega: ZIP del proyecto Rascal + este documento explicativo

Resumen ejecutivo

Este documento describe la version final del Proyecto 3 de VeriLang. La solucion parte de la definicion del lenguaje construida en Rascal y la completa con parser, AST, generador, integracion de TypePal, anotaciones de tipos, chequeo semantico y pruebas validas e invalidas. Esta version tambien documenta los ultimos cambios realizados en el ZIP final: soporte para los operadores relacionales < y >, correccion de la definicion lexica de INT, validacion de operadores infijos personalizados, validacion de reglas defrule, verificacion de tipos de usuario mediante defspace, y bloqueo de la generacion cuando existen errores semanticos.

Tabla de contenido

1. Introduccion y objetivo
2. Cumplimiento del enunciado
3. Como se construyo la solucion
4. Correcciones de la iteracion anterior y cambios finales
5. Estructura del proyecto y archivos incluidos
6. Descripcion archivo por archivo
7. Gramatica concreta y tipos
8. Sintaxis abstracta e Implode
9. Parser del lenguaje
10. Generador de codigo
11. Integracion de TypePal en Main.rsc y Plugin.rsc
12. Sistema de tipos y reglas semanticas
13. Regla nueva de existencia de estructuras/tipos
14. Archivos de prueba y salidas esperadas
15. Como probar el proyecto
16. Comentario sobre mensajes informativos de TypePal
17. Fuente de la solucion
18. Conclusiones

1. Introduccion y objetivo

El objetivo del Proyecto 3 fue reutilizar la definicion de VeriLang implementada previamente en Rascal y construir una version mas completa del lenguaje. A partir de la gramatica y la sintaxis abstracta de la segunda parte, se agregaron anotaciones de tipo, un sistema de tipos con TypePal, generacion de codigo y un flujo principal de ejecucion sobre archivos .vl.

La version final permite tomar un programa VeriLang escrito en un archivo con extension .vl, parsearlo, construir su representacion, validar sus nombres y tipos, y finalmente generar una version textual del programa en consola. Si TypePal encuentra errores semanticos, el flujo principal reporta los errores y no genera codigo.

La entrega tambien incluye ejemplos validos y ejemplos invalidos para demostrar que el lenguaje acepta programas correctos y detecta errores de tipos, operadores, reglas y dominios de usuario no declarados.

2. Cumplimiento del enunciado

#	Requisito	Evidencia en la solucion	Estado
1	Definir el parser del lenguaje	Parser.rsc expone parseMainModule y usa la gramatica concreta de Syntax.rsc.	Cumplido
2	Generar codigo desde sintaxis y AST usando entrada .vl y salida en consola	Main.rsc recibe un loc de archivo .vl, llama a Generator.rsc e imprime el resultado generado.	Cumplido
3	Instalar TypePal en el lenguaje	Checker.rsc extiende analysis::typepal::TypePal y RASCAL.MF declara lib://typepal.	Cumplido
4	Agregar anotaciones de tipo para valores y estructuras	Domain soporta int, bool, real, string, char y tipos de usuario por ID.	Cumplido
5	Verificar que los tipos usados correspondan a sus definiciones	Checker.rsc valida relaciones, and/or/not, operadores infijos, invocaciones y defrule.	Cumplido
6	Definir una regla nueva para asegurar existencia de elementos en estructuras	La regla implementada exige que todo tipo/dominio de usuario usado exista previamente como defspace.	Cumplido
Doc.	Documento explicativo	Este documento explica construccion, archivos, cambios, fuente, funcionamiento y pruebas.	Cumplido

3. Como se construyo la solucion

La solucion se construyo de forma incremental a partir del proyecto propio desarrollado en las iteraciones anteriores de VeriLang. Primero se corrigieron los problemas de base que impedian importar o usar ciertos modulos. Luego se extendio la gramatica para soportar tipos, se actualizo el AST, se ajusto el generador y finalmente se integro TypePal para validar semanticamente los programas.

3.1 Proceso general de construccion

- Se partio de la gramatica concreta de VeriLang en Syntax.rsc.
- Se corrigieron errores de la iteracion anterior, como el campo duplicado en Number y problemas de cuantificadores.
- Se agrego Domain para representar tipos primitivos y tipos definidos por usuario.
- Se actualizo AST.rsc para que la sintaxis abstracta correspondiera con la gramatica concreta.
- Se mantuvo Implode.rsc como puente entre el arbol concreto y el AST.
- Se actualizo Generator.rsc para producir texto VeriLang desde el AST.
- Se creo Checker.rsc con TypePal, tipos propios del lenguaje, roles de identificadores y reglas semanticas.
- Se integro Checker.rsc en Main.rsc para que la ejecucion principal valide antes de generar.
- Se ajusto Plugin.rsc para que el entorno tambien bloquee la generacion cuando existen errores semanticos.
- Se agregaron pruebas validas e invalidas dentro de instance/.

3.2 Flujo interno del lenguaje

- El usuario entrega o selecciona un archivo .vl.
- Parser.rsc construye un arbol concreto a partir de Syntax.rsc.

- Checker.rsc construye un modelo semantico con TypePal.
- TypePal resuelve definiciones, usos y tipos.
- Main.rsc filtra los mensajes de tipo error.
- Si hay errores, se imprimen y se detiene la generacion.
- Si no hay errores, Generator.rsc convierte el programa a texto VeriLang.
- El resultado generado se muestra en consola.

4. Correcciones de la iteracion anterior y cambios finales

Ademas de implementar los requisitos nuevos del Proyecto 3, se corrigieron problemas de la iteracion anterior para evitar que el lenguaje siguiera fallando por errores ya reportados. Tambien se incorporaron ajustes finales del ZIP actual para mantener consistencia entre Syntax.rsc, AST.rsc, Generator.rsc, Checker.rsc, Main.rsc y Plugin.rsc.

4.1 Correccion del tipo Number en AST.rsc

El AST anterior reutilizaba el nombre de campo val para enteros y reales dentro del mismo tipo algebraico Number. Rascal no permite declarar el mismo nombre de campo con tipos distintos en alternativas del mismo tipo. La version final usa nombres separados.

```
data Number
  = intNumber(int valInt)
  | floatNumber(num valFloat)
;
```

4.2 Correccion de Plugin.rsc y RASCAL.MF

Se mantuvo el nombre del proyecto como rascaldslverilang y se declaro TypePal como libreria requerida. Esto permite que el plugin y el proyecto encuentren correctamente las rutas y dependencias.

```
Manifest-Version: 0.0.1
Project-Name: rascaldslverilang
Source: src/main/rascal
Require-Libraries: |lib://rascal|,|lib://typepal|
```

4.3 Correccion de imports y cuantificadores

La regla de imports se mantuvo simple para no exigir saltos de linea artificiales. Los cuantificadores se corrigieron para exigir siempre un cuerpo despues del punto, evitando formas ambiguas o incompletas.

```
syntax FileImport
  = fileImport: 'using' ID importName
;

syntax TopExp
  = quantExp: '(' Quantifier quantifier ID obj1 'in' ID obj2 '.' TopExp topExp ')'
  | orExpRec: OrExp orExp
;
```

4.4 Correccion del ejemplo principal spec1.vl

Cuando se corrigio el chequeo de operadores infijos, el cuerpo del cuantificador no podia seguir siendo solo (b inter a), porque inter retorna Set y el cuerpo de un cuantificador debe ser booleano. Por eso el ejemplo principal se actualizo para comparar el resultado contra a.

```
defoperator inter : Set -> Set -> Set end
defexpression (forall x in Set . ((b inter a) = a)) end
```

4.5 Soporte final para < y >

En el ZIP final se completo la consistencia de operadores relacionales simples. AST.rsc y Generator.rsc ya contemplaban gt() y lt(), por lo que Syntax.rsc se actualizo para reconocer tambien > y <.

```
syntax RelOp
  = eq: '='
  | gt: '\>'
  | lt: '\<'
  | ge: '\>='
  | le: '\<='
  | equiv: '≡'
```

```
| iff: '\<\>'
;
```

4.6 Correccion de INT

La definicion lexic de INT se corrigio para aceptar enteros positivos y negativos correctamente, evitando que el simbolo - aislado fuera reconocido como entero.

```
lexical INT = [\-]? [0-9]+ !>> [0-9];
```

4.7 Plugin.rsc tambien valida antes de generar

Ademas de Main.rsc, Plugin.rsc ejecuta TypePal antes de generar archivos desde el entorno. Si encuentra errores semanticos, escribe un reporte de errores y no genera codigo VeriLang.

5. Estructura del proyecto y archivos incluidos

La estructura final separa definicion del lenguaje, ejemplos validos, ejemplos invalidos, salidas generadas y configuracion del proyecto Rascal. Los archivos de target/ y .vscode/ pueden aparecer en el ZIP como archivos generados o de configuracion local, pero no son la parte central de la implementacion.

```
rascaldslverilang/
  META-INF/
    RASCAL.MF
  src/main/rascal/
    AST.rsc
    Checker.rsc
    Generator.rsc
    Implode.rsc
    Main.rsc
    Parser.rsc
    Plugin.rsc
    Syntax.rsc
  instance/
    spec1.vl
    spec2.vl
    spec3.vl
    spec_types.vl
    errors/
      spec_type_error.vl
      spec_unknown_space_error.vl
      spec_logic_error.vl
      spec_operator_error.vl
      spec_rule_error.vl
    output/
      testGenerator.vl
      testGenerator2.vl
  pom.xml
  .vscode/               configuracion local opcional
  target/                archivos generados por build
```

6. Descripcion archivo por archivo

Archivo	Por que existe / funcion
Syntax.rsc	Define la gramatica concreta de VeriLang: modulos, imports, espacios, operadores, variables, reglas, expresiones, tipos, literales y operadores relacionales.
AST.rsc	Define la sintaxis abstracta mediante tipos algebraicos de Rascal. Incluye MainModule, Statement, Domain, Invocation, TopExp, Primary, RelOp y literales.
Parser.rsc	Expone parseMainModule para parsear codigo VeriLang desde texto o desde un archivo .vl.
Implode.rsc	Convierte el arbol concreto producido por el parser en el AST definido en AST.rsc usando implode.
Generator.rsc	Recorre el AST y vuelve a generar codigo textual VeriLang. Tambien tiene un main de prueba que escribe testGenerator2.vl.
Checker.rsc	Implementa TypePal, tipos semanticos, roles de identificadores, definiciones, usos y reglas de chequeo de tipos.
Main.rsc	Ejecuta el flujo principal: parsear archivo .vl, correr TypePal, imprimir errores si existen y generar codigo en consola si el programa es valido.

Plugin.rsc	Registra el lenguaje VL en el entorno, agrega un lente para generar código y bloquea la generación si hay errores semánticos.
RASCAL.MF	Declara el nombre del proyecto, la ruta fuente y las librerías requeridas: rascal y typepal.
pom.xml	Configuración Maven del proyecto Rascal.
instance/spec*.vl	Programas válidos usados para probar parser, generador y checker.
instance/errors/*.vl	Programas inválidos usados para demostrar errores semánticos esperados.
instance/output/*.vl	Archivos de salida generados o conservados como evidencia de generación.

7. Gramática concreta y tipos

La gramática concreta se encuentra en Syntax.rsc. La regla central para el sistema de tipos es Domain, que reconoce tipos primitivos y tipos definidos por el usuario. Los tipos de usuario se representan con ID y deben existir como defspace cuando se usan como dominio.

```
syntax Domain
  = boolDomain: 'bool'
  | intDomain: 'int'
  | realDomain: 'real'
  | stringDomain: 'string'
  | charDomain: 'char'
  | nameDomain: ID domainName
;
```

Las reglas de frule usan invocaciones unarias y binarias separadas. Esta decisión facilita validar aridad y tipos de argumentos en Checker.rsc.

```
syntax Invocation
  = unaryInvocation: '(' ID opName Primary param ')'
  | binaryInvocation: '(' ID opName Primary param1 Primary param2 ')'
;
```

Los operadores relacionales reconocidos incluyen igualdad, mayor que, menor que, mayor o igual, menor o igual, equivalencia e iff. Esto mantiene alineados Syntax.rsc, AST.rsc y Generator.rsc.

8. Sintaxis abstracta e Implode

AST.rsc define la representación abstracta que usan el generador y el checker. Por ejemplo, Invocation se separa en dos constructores concretos para distinguir reglas unarias y binarias.

```
data Invocation
  = unaryInvocation(str opName, Primary param)
  | binaryInvocation(str opName, Primary param1, Primary param2)
;
```

Implode.rsc conserva una implementación simple porque las etiquetas de Syntax.rsc y AST.rsc están alineadas. Por eso Rascal puede convertir automáticamente el árbol concreto al AST.

```
public MainModule implodeMain(Tree pt) = implode(#MainModule, pt);
public MainModule load(loc l) = implodeMain(parseMainModule(l));
```

9. Parser del lenguaje

Parser.rsc expone dos formas de parsear: una desde texto fuente y otra desde una ubicación loc. Esto permite probar programas desde la terminal de Rascal y ejecutar archivos .vl del proyecto.

```
public start[MainModule] parseMainModule(str src, loc origin)
  = parse(#start[MainModule], src, origin);

public start[MainModule] parseMainModule(loc origin)
  = parse(#start[MainModule], origin);
```

Ejemplos válidos de parseo:

```
parseMainModule(|project://rascaldslverilang/instance/spec1.vl|);
parseMainModule(|project://rascaldslverilang/instance/spec2.vl|);
```

```

parseMainModule(|project://rascaldslverilang/instance/spec3.vl|);
parseMainModule(|project://rascaldslverilang/instance/spec_types.vl|);

```

10. Generador de codigo

Generator.rsc recibe un arbol concreto, lo convierte al AST con implodeMain y despues produce texto VeriLang. El resultado principal del generador se usa desde Main.rsc para imprimir en consola.

```

str generator(Tree cst) {
    MainModule ast = implodeMain(cst);
    return generate(ast);
}

```

Despues del cambio de Invocation, el generador incluye funciones separadas para invocaciones unarias y binarias.

```

str generate(unaryInvocation(str name, Primary param)) {
    return "<name> <generate(param)>";
}

str generate(binaryInvocation(str name, Primary param1, Primary param2)) {
    return "<name> <generate(param1)> <generate(param2)>";
}

```

Main.rsc no depende de escribir archivos de salida: su responsabilidad principal es mostrar el resultado en consola, como pide el enunciado. Generator.rsc y Plugin.rsc pueden producir archivos en instance/output como evidencia adicional o apoyo del entorno.

11. Integracion de TypePal en Main.rsc y Plugin.rsc

11.1 Main.rsc

Main.rsc implementa el flujo principal. Primero parsea el archivo .vl, luego ejecuta TypePal y filtra mensajes de tipo error. Si hay errores, los muestra y retorna 1. Si no hay errores, genera codigo y retorna 0.

```

Tree cst = parseMainModule(archivo);
TModel tm = modulesTModelFromTree(cst);

list[Message] errors = [
    m | m <- getMessages(tm), m is error
];

if (errors != []) {
    for (m <- errors) {
        println(m);
    }
    return 1;
}

str result = generator(cst);
println(result);
return 0;

```

11.2 Plugin.rsc

Plugin.rsc registra el lenguaje VL y agrega un lente de generacion. En la version final tambien ejecuta el checker antes de generar: si hay errores, escribe generator_errors.txt y no produce codigo generado.

```

TModel tm = modulesTModelFromTree(cst);
list[Message] errors = [m | m <- getMessages(tm), m is error];

if (errors != []) {
    writeFile(|project://rascaldslverilang/instance/output/generator_errors.txt|, report);
    return ("result": false);
}

str rVal = generator(cst);
writeFile(|project://rascaldslverilang/instance/output/generator.vl|, rVal);

```

12. Sistema de tipos y reglas semanticas

Checker.rsc define los tipos semanticos del lenguaje con AType. Los tipos primitivos se representan directamente y los tipos definidos por usuario se representan como userType(str name). Los operadores se modelan con functionType, donde la lista guarda argumentos y tipo de retorno.

```
data AType
  = boolType()
  | intType()
  | realType()
  | stringType()
  | charType()
  | userType(str name)
  | functionType(list[AType] signature)
;
```

Tambien se definen roles de identificadores. Esto permite diferenciar variables, operadores y espacios definidos por el usuario.

```
data IdRole
  = variableId()
  | operatorId()
  | spaceId()
;
```

12.1 Variables y dominios

Las variables se registran con el tipo indicado en su declaracion. Si el dominio es un tipo de usuario, Checker.rsc exige que exista un defspace correspondiente.

12.2 Relaciones

Para relaciones como =, <, >, <=, >=, $\equiv$ y <>, el checker exige que ambos lados tengan el mismo tipo. El resultado de una relacion es boolType().

12.3 Operadores logicos

Los operadores and, or y not se validan con TypePal. and y or requieren operandos booleanos. not solo puede aplicarse a una expresion booleana.

12.4 Operadores infijos personalizados

Los operadores infijos personalizados ya no se consideran booleanos automaticamente. La version final consulta la firma declarada del operador, valida el tipo del lado izquierdo, valida el tipo del lado derecho y conserva el tipo real de retorno.

```
defoperator belongsTo : Person -> Group -> bool end
defexpression age belongsTo name end

-- Si age:int y name:string, el checker reporta errores porque
-- la firma espera Person y Group.
```

12.5 Reglas defrule

Las reglas defrule se validan para invocaciones unarias y binarias. El checker revisa aridad, tipos de argumentos y que ambos lados de la regla produzcan el mismo tipo.

```
defoperator suma : int -> int -> int end
defrule (suma x b) -> (suma b x) end

-- Si x:int y b:bool, la regla es invalida porque suma espera int e int.
```

13. Regla nueva de existencia de estructuras/tipos

El enunciado pide definir una regla nueva para asegurar que los elementos usados en una estructura de datos existan. En esta version de VeriLang, las estructuras/dominios definidos por el usuario se modelan mediante defspace. Por eso, la regla implementada es: todo dominio o estructura de usuario que se use como tipo debe existir previamente como defspace.

Uso valido:


```

defspace Person end

defvar
  p : Person
end

```

Uso invalido:

```

defvar
  x : UnknownSpace
end

```

La regla se implementa usando `c.use` sobre dominios de usuario. Si el tipo no existe como `spaceId`, `TypePal` reporta un error de espacio indefinido.

```

if (domainToType(d) is userType) {
  c.use(d, {spaceId()});
}

```

14. Archivos de prueba y salidas esperadas

Archivo	Tipo	Proposito
instance/spec1.vl	Valido	Programa principal: spaces, variables, operadores, reglas, atributos, cuantificadores y expresiones.
instance/spec2.vl	Valido	Prueba variables, union, igualdad y expresiones basicas.
instance/spec3.vl	Valido	Prueba regla unaria con negation.
instance/spec_types.vl	Valido	Prueba int, bool, real, string, char y tipos de usuario Person/Group.
instance/errors/spec_type_error.vl	Invalido	Detecta comparacion int vs bool.
instance/errors/spec_unknown_space_error.vl	Invalido	Detecta tipo de usuario no declarado como defspace.
instance/errors/spec_logic_error.vl	Invalido	Detecta uso de and con un operando no booleano.
instance/errors/spec_operator_error.vl	Invalido	Detecta operador infijo usado con argumentos incorrectos.
instance/errors/spec_rule_error.vl	Invalido	Detecta argumentos incorrectos dentro de defrule.

14.1 Pruebas validas

En los archivos validos no deben aparecer mensajes `error(...)`. Pueden aparecer mensajes `info(...)` de `TypePal`, pero no representan errores semanticos reales del programa `VeriLang`.

```

getMessages(modulesTModelFromTree(parseMainModule(|project://rascaldslverilang/instance/spec1.vl|)));
getMessages(modulesTModelFromTree(parseMainModule(|project://rascaldslverilang/instance/spec2.vl|)));
getMessages(modulesTModelFromTree(parseMainModule(|project://rascaldslverilang/instance/spec3.vl|)));
getMessages(modulesTModelFromTree(parseMainModule(|project://rascaldslverilang/instance/spec_types.vl|)));

```

14.2 Pruebas invalidas

```

getMessages(modulesTModelFromTree(parseMainModule(|project://rascaldslverilang/instance/errors/spec_type_error.vl|)));
-- Esperado: Los dos lados de la relacion deben tener el mismo tipo

getMessages(modulesTModelFromTree(parseMainModule(|project://rascaldslverilang/instance/errors/spec_unknown_space_error.vl|)));
-- Esperado: Undefined space `UnknownSpace`

getMessages(modulesTModelFromTree(parseMainModule(|project://rascaldslverilang/instance/errors/spec_logic_error.vl|)));
-- Esperado: El lado izquierdo de and debe tener tipo booleano

getMessages(modulesTModelFromTree(parseMainModule(|project://rascaldslverilang/instance/errors/spec_operator_error.vl|)));
-- Esperado: El lado izquierdo/derecho del operador no coincide con su firma

getMessages(modulesTModelFromTree(parseMainModule(|project://rascaldslverilang/instance/errors/spec_rule_error.vl|)));
-- Esperado: argumentos de suma no coinciden con el tipo esperado

```

15. Como probar el proyecto

15.1 Importar modulos principales

```
import AST;
import Syntax;
import Implode;
import Parser;
import Checker;
import Generator;
import Main;
import Plugin;
```

Cada import debe cargar correctamente. Si TypePal no esta disponible, se debe revisar RASCAL.MF y la instalacion del proyecto.

15.2 Ejecutar programa valido desde Main.rsc

```
import Main;
Main::main();

-- Usa por defecto:
-- |project://rascaldslverilang/instance/spec1.vl|
```

Resultado esperado para un programa valido: el parser se ejecuta correctamente, el checker no encuentra errores semanticos, el generador produce codigo VeriLang y Main.rsc retorna int: 0.

15.3 Ejecutar programa invalido desde Main.rsc

```
Main::main(|project://rascaldslverilang/instance/errors/spec_rule_error.vl|);
```

Resultado esperado para un programa invalido: Main.rsc muestra los errores semanticos, no genera codigo y retorna int: 1.

15.4 Probar el plugin

```
import Plugin;
Plugin::main();

-- Resultado esperado:
-- Plugin loaded!
```

Desde el entorno, el lente de generacion produce codigo solo si el programa no tiene errores semanticos. Si hay errores, se genera un reporte en instance/output/generator_errors.txt.

16. Comentario sobre mensajes informativos de TypePal

Durante algunas pruebas TypePal puede producir mensajes info, por ejemplo Missing collect for nodos envoltorio. Estos mensajes no son errores semanticos del programa VeriLang. Por esa razon Main.rsc y Plugin.rsc filtran los mensajes y toman decisiones solo con mensajes de tipo error.

```
list[Message] errors = [
  m | m <- getMessages(tm), m is error
];
```

La decision de no bloquear por mensajes info evita detener la generacion por advertencias internas de TypePal que no representan errores del lenguaje del usuario.

17. Fuente de la solucion

La solucion parte del proyecto propio desarrollado en las iteraciones anteriores de VeriLang. No se tomo una solucion externa completa como base. La documentacion y la gramatica previas sirvieron como punto de partida conceptual, pero los cambios finales fueron implementados sobre los archivos propios del proyecto.

Los cambios principales se realizaron en Syntax.rsc, AST.rsc, Implode.rsc, Generator.rsc, Checker.rsc, Main.rsc y Plugin.rsc. Tambien se agregaron y organizaron archivos de prueba validos e invalidos dentro de instance/.

18. Conclusiones

La version final de VeriLang cumple el flujo central solicitado para el Proyecto 3: parsear archivos .vl, representar el programa, generar codigo y ejecutar verificaciones semanticas con TypePal. El sistema reconoce tipos primitivos, tipos definidos por usuario, operadores logicos, relaciones, operadores infijos personalizados, invocaciones en reglas y dominios de usuario.

La integracion de TypePal en Main.rsc es esencial porque impide generar codigo cuando existen errores semanticos. Ademas, Plugin.rsc replica este comportamiento en el entorno, de modo que la generacion desde el lente tambien respeta el checker.

Los ejemplos validos e invalidos incluidos en instance/ permiten comprobar que el checker acepta programas correctos y reporta errores en programas incorrectos. Con esto se demuestra el cumplimiento de los requisitos del enunciado y se documenta claramente como se construyo, como funciona y como se prueba la solucion.